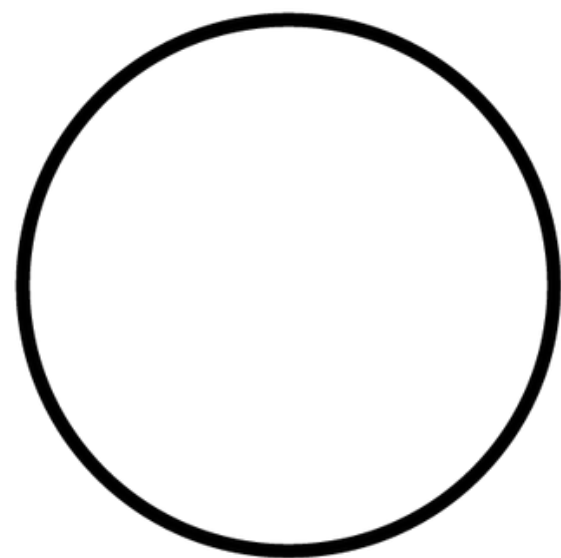


CLASE 4

BÚSQUEDA EN ANCHURA (BFS)

¿QUÉ ES UN GRAFO?

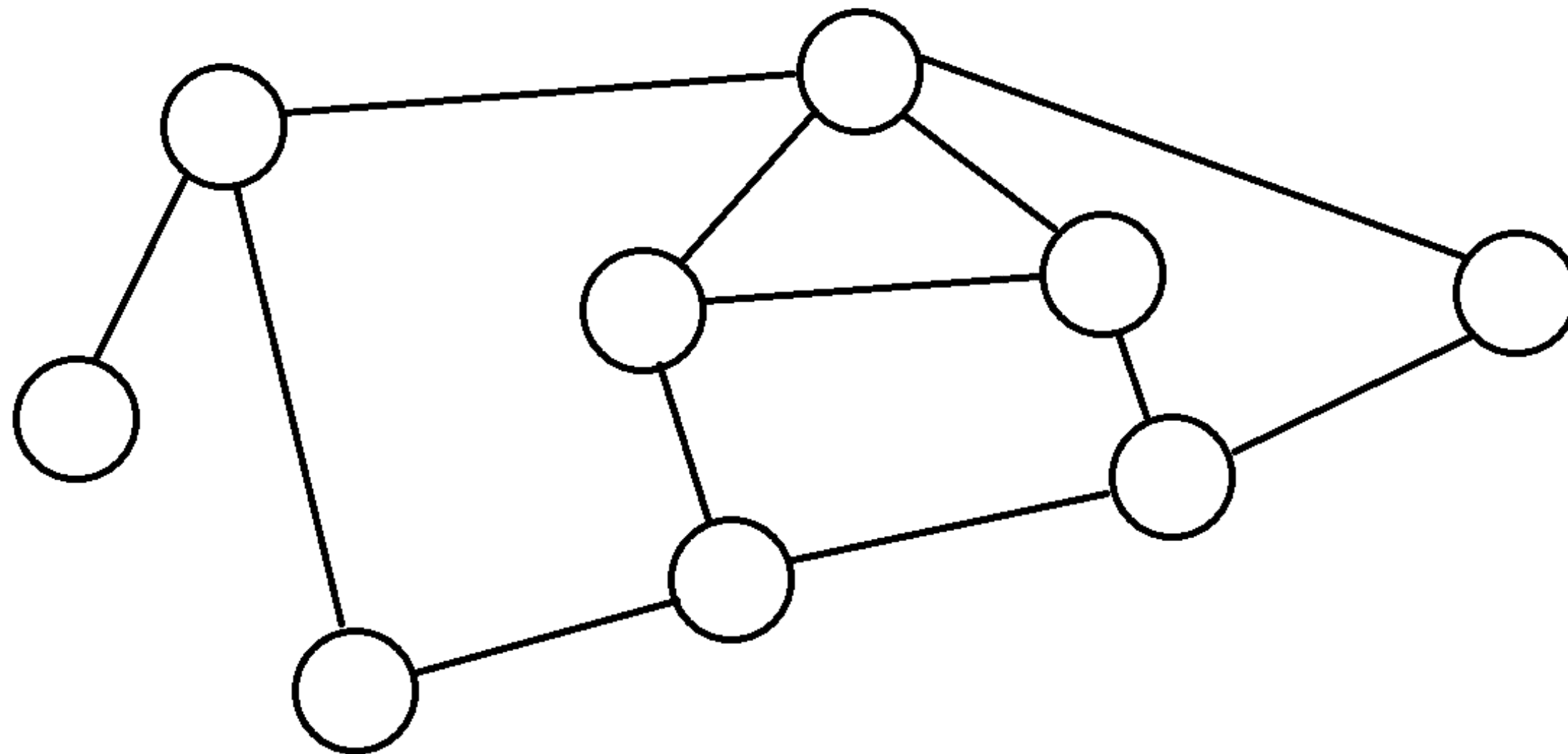


NODOS



ARISTAS

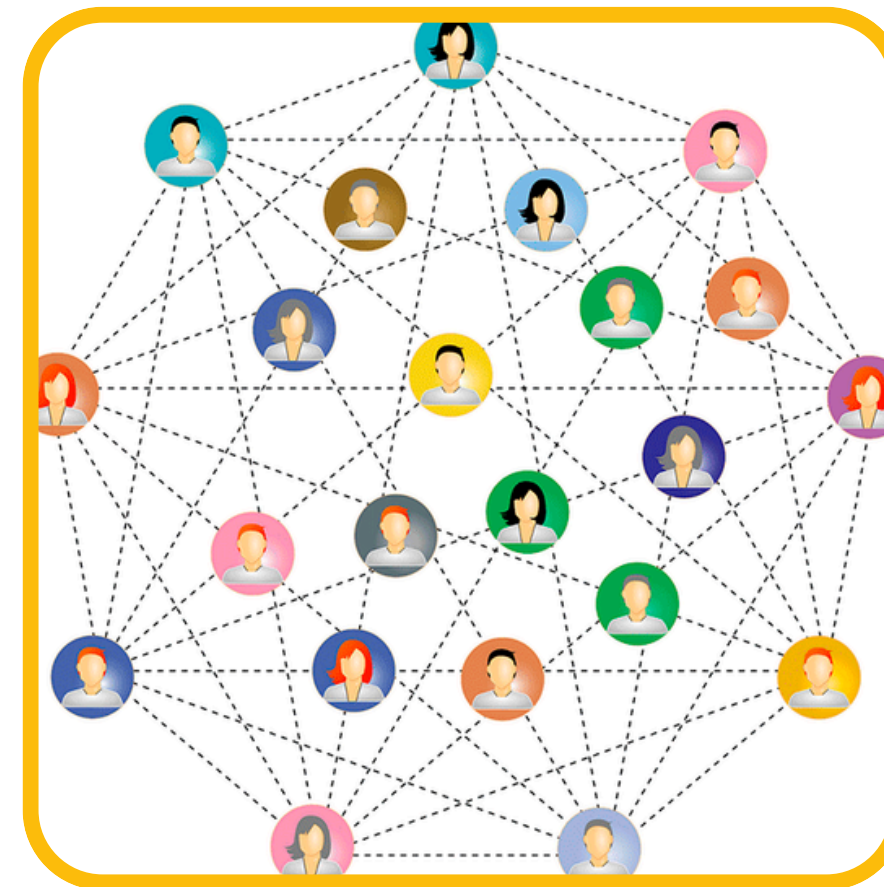
NODOS CONECTADOS POR ARISTAS



¿PARA QUE SIRVEN?



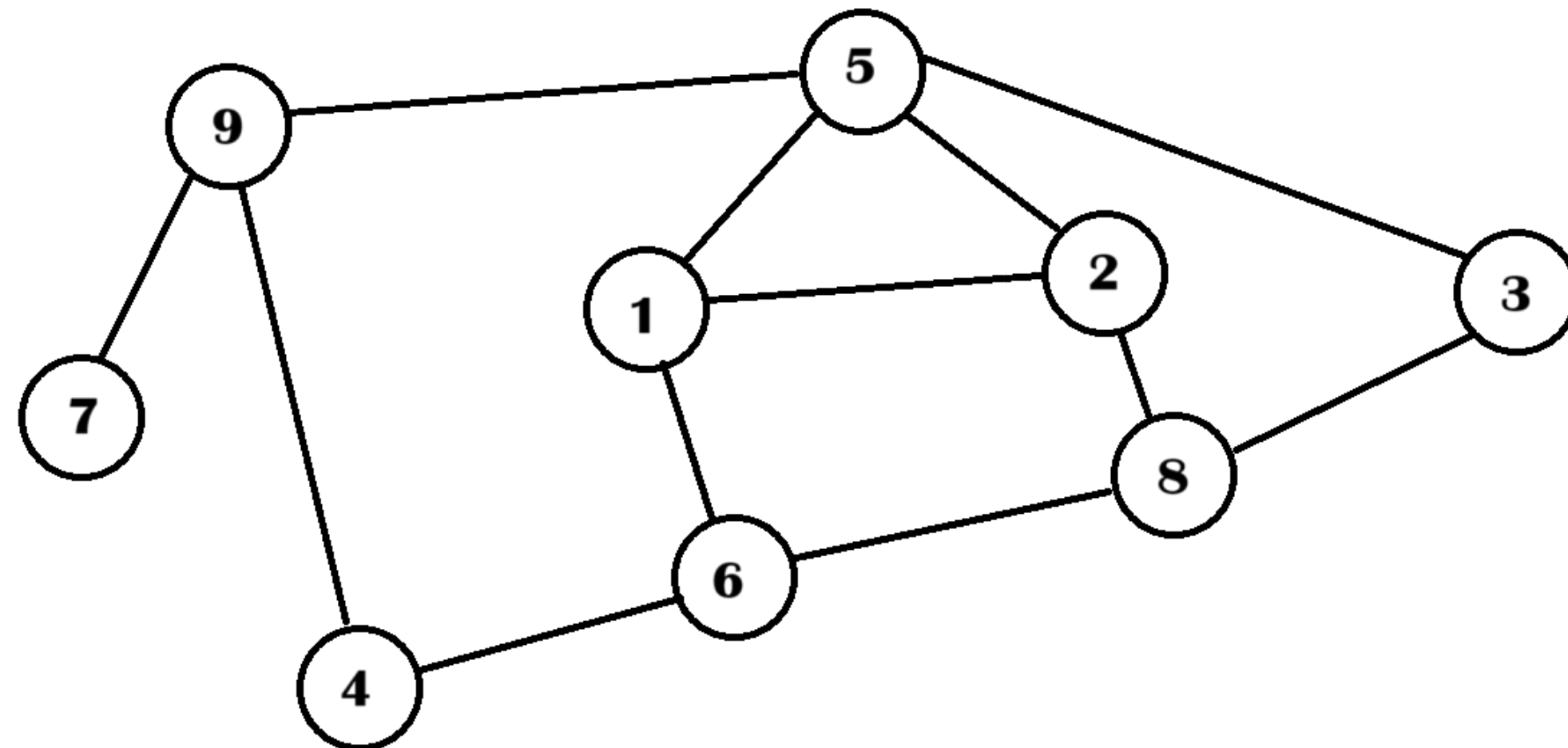
RED DE METRO



REDES SOCIALES

COMO REPRESENTARLOS

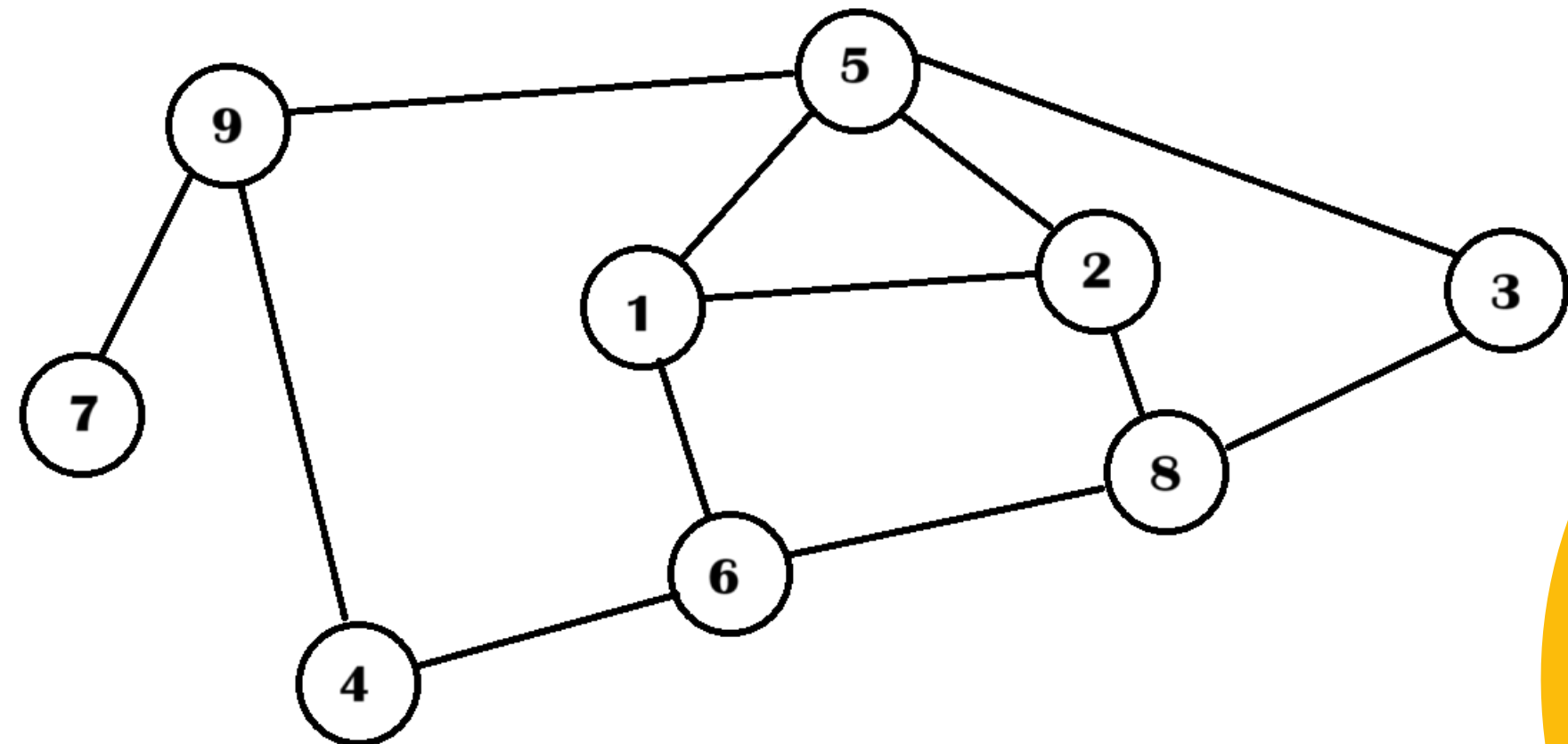
Dibujarlos es sencillo, pero representarlos en el computador es más desafiante. Primero, enumeremos los nodos (arbitrariamente)



ENUMERAR ARISTAS

Una forma de guardar nuestro grafo es guardar todas las aristas presentes en el grafo. Nos permite poder recorrer todas las aristas, pero si estoy en un nodo puede ser difícil acceder a sus aristas

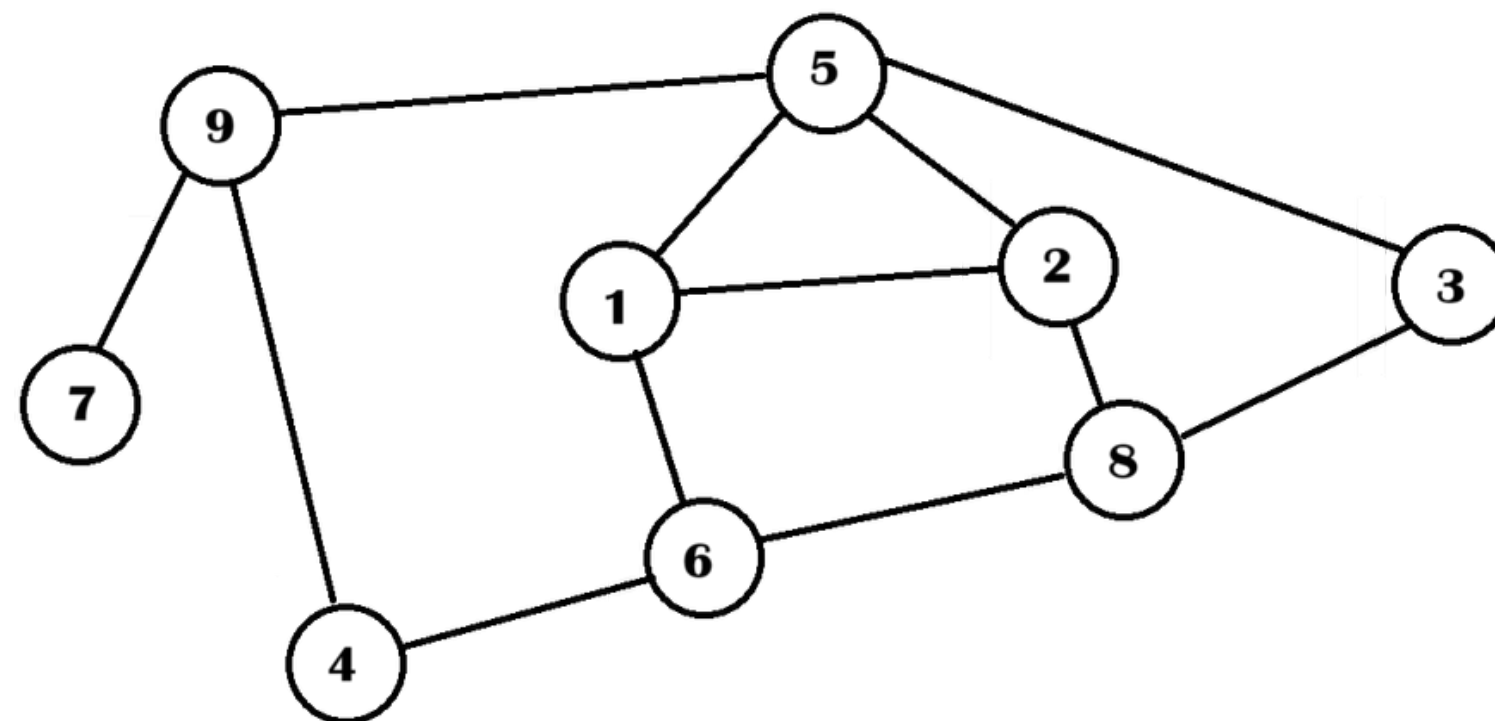
- (1, 2)
- (1, 5)
- (1, 6)
- (2, 5)
- (2, 8)
- (3, 5)
- (3, 8)
- (4, 6)
- (4, 9)
- (5, 9)
- (6, 8)
- (7, 9)



MATRIZ DE ADYACENCIA

Tenemos una matriz de $n \times n$, en la que hay un 1 en la posición (i, j) si existe una arista entre i y j , y 0 en caso contrario. La ventaja es que nos permite saber si hay una arista rápidamente, pero la desventaja es que usamos $O(n^2)$ memoria.

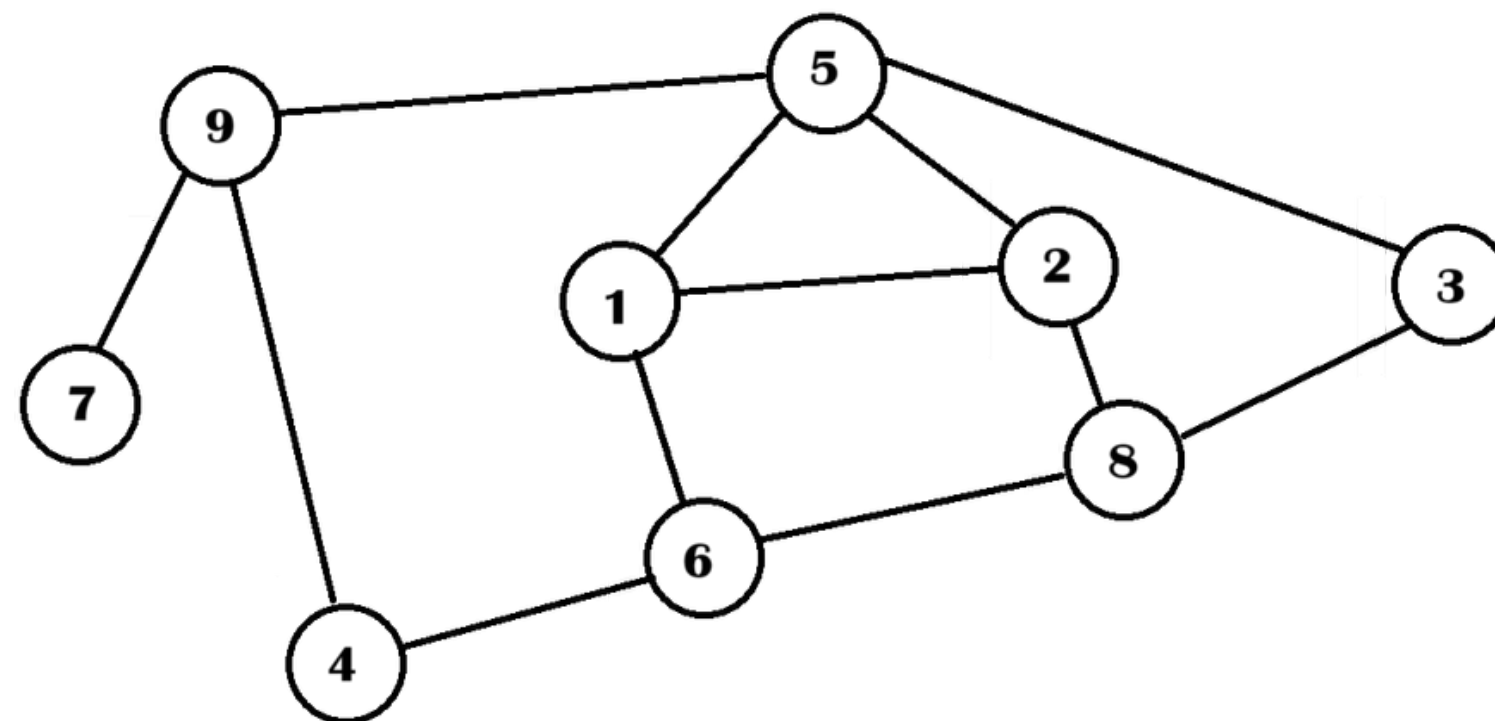
0	1	0	0	1	1	0	0	0
1	0	0	0	1	0	0	1	0
0	0	0	0	1	0	0	1	0
0	0	0	0	0	1	0	0	1
1	1	1	0	0	0	0	0	1
1	0	0	1	0	0	0	1	0
0	0	0	0	0	0	0	0	1
0	1	1	0	0	1	0	0	0
0	0	0	1	1	0	1	0	0



LISTA DE ADYACENCIA

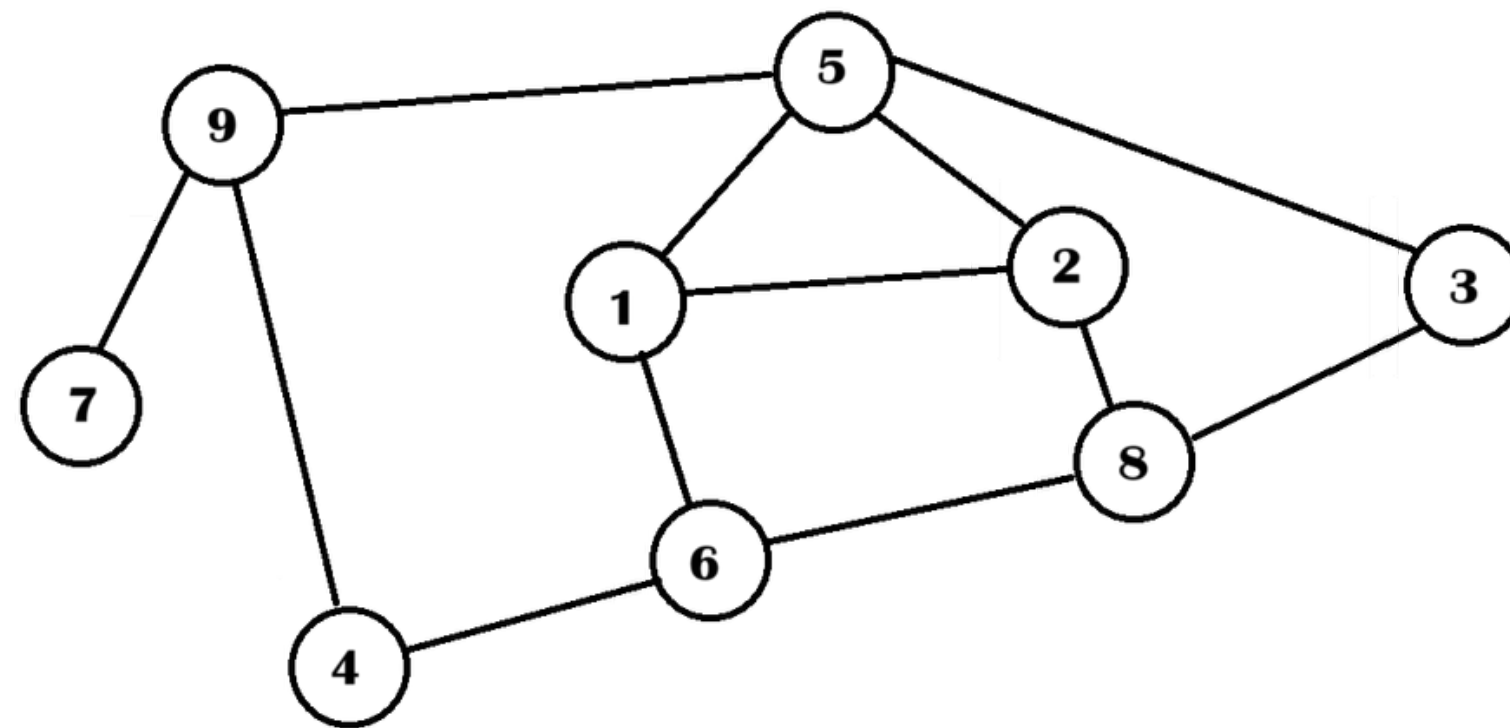
Guardamos para cada nodo una lista con los nodos que son adyacentes a él. Esta es muy conveniente ya que nos permite iterar sobre las aristas y acceder a las aristas de un nodo en particular.

1: [2, 5, 6]
2: [1, 5, 8]
3: [5, 8]
4: [6, 9]
5: [1, 2, 3, 9]
6: [1, 4, 8]
7: [9]
8: [2, 3, 6]
9: [4, 5, 7]



PROBLEMA

Dado un grafo, queremos encontrar la distancia desde el nodo 1 hacia todo el resto de nodos.



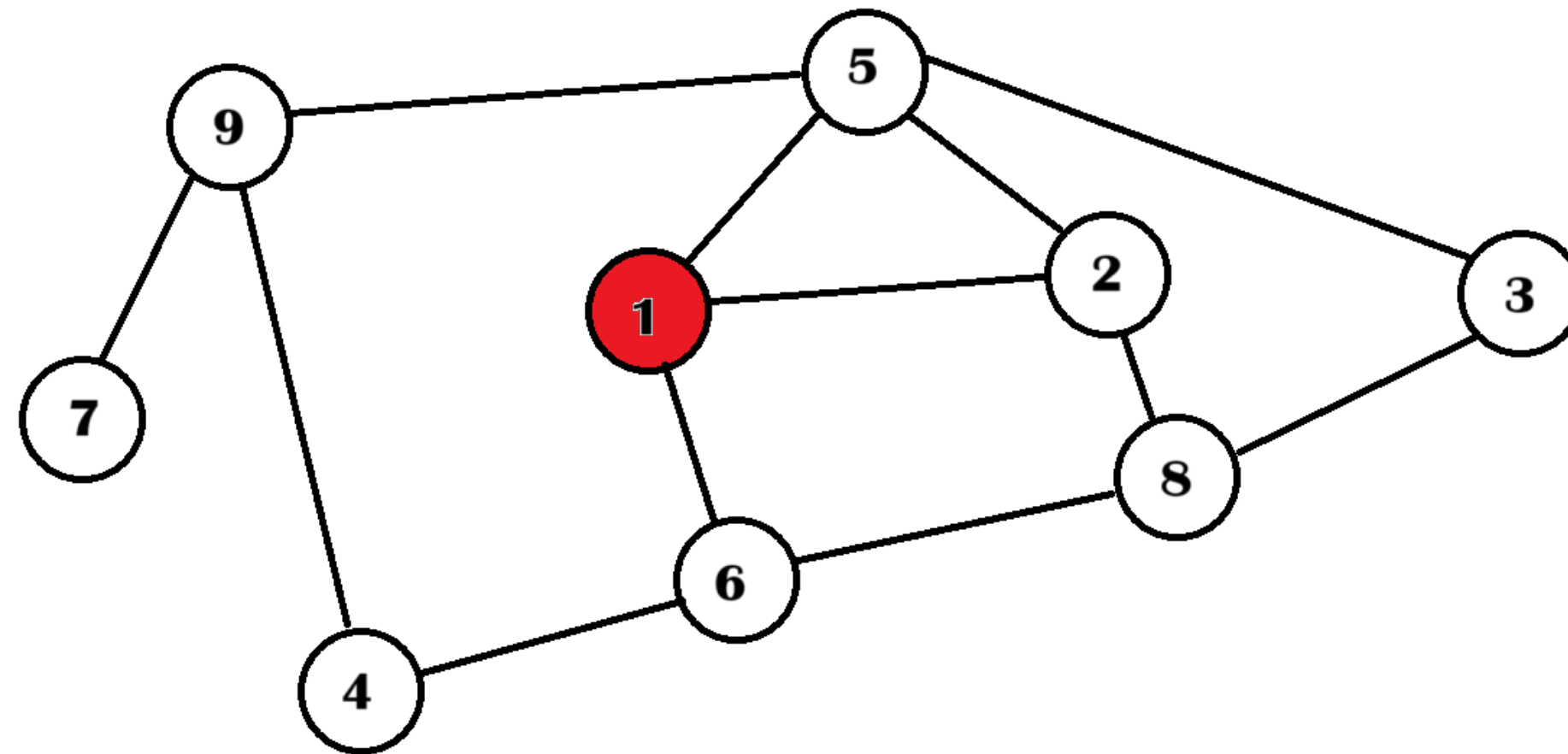
Por ejemplo, aquí la distancia sería $d = [0, 1, 2, 2, 1, 1, 3, 2, 2]$

OBSERVACIONES

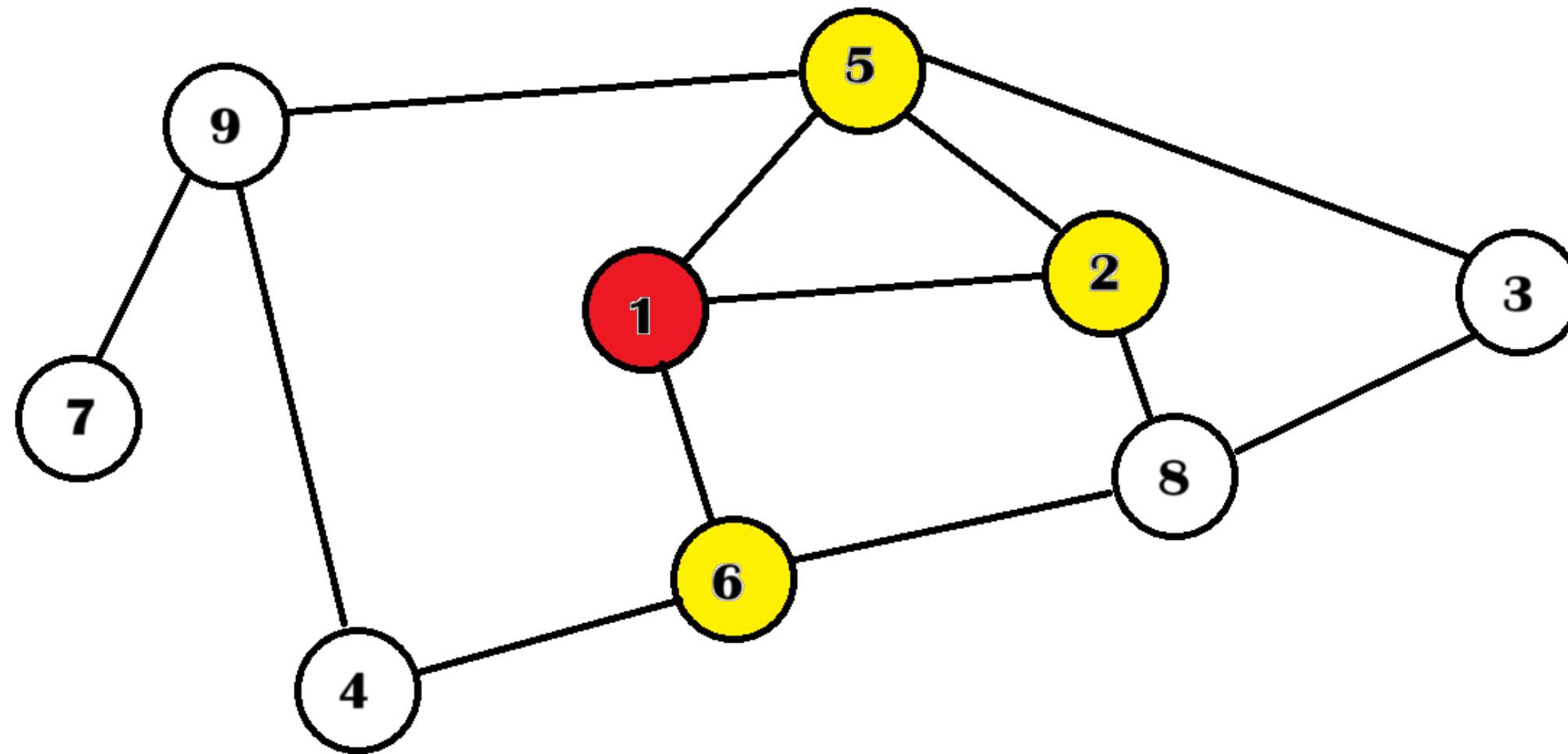
- Inicialmente para el único nodo que conozco su distancia es para el inicial, la cual es 0.
- Si un nodo tiene distancia x , entonces debe tener un vecino con distancia $x-1$ desde el que llegué.
- Si a un nodo ya encontré como llegar con una distancia x , encontrar otra forma de llegar con una distancia mayor que x no me aporta nada.
- La máxima distancia es a lo más $n-1$.

SOLUCIÓN CONCEPTUAL

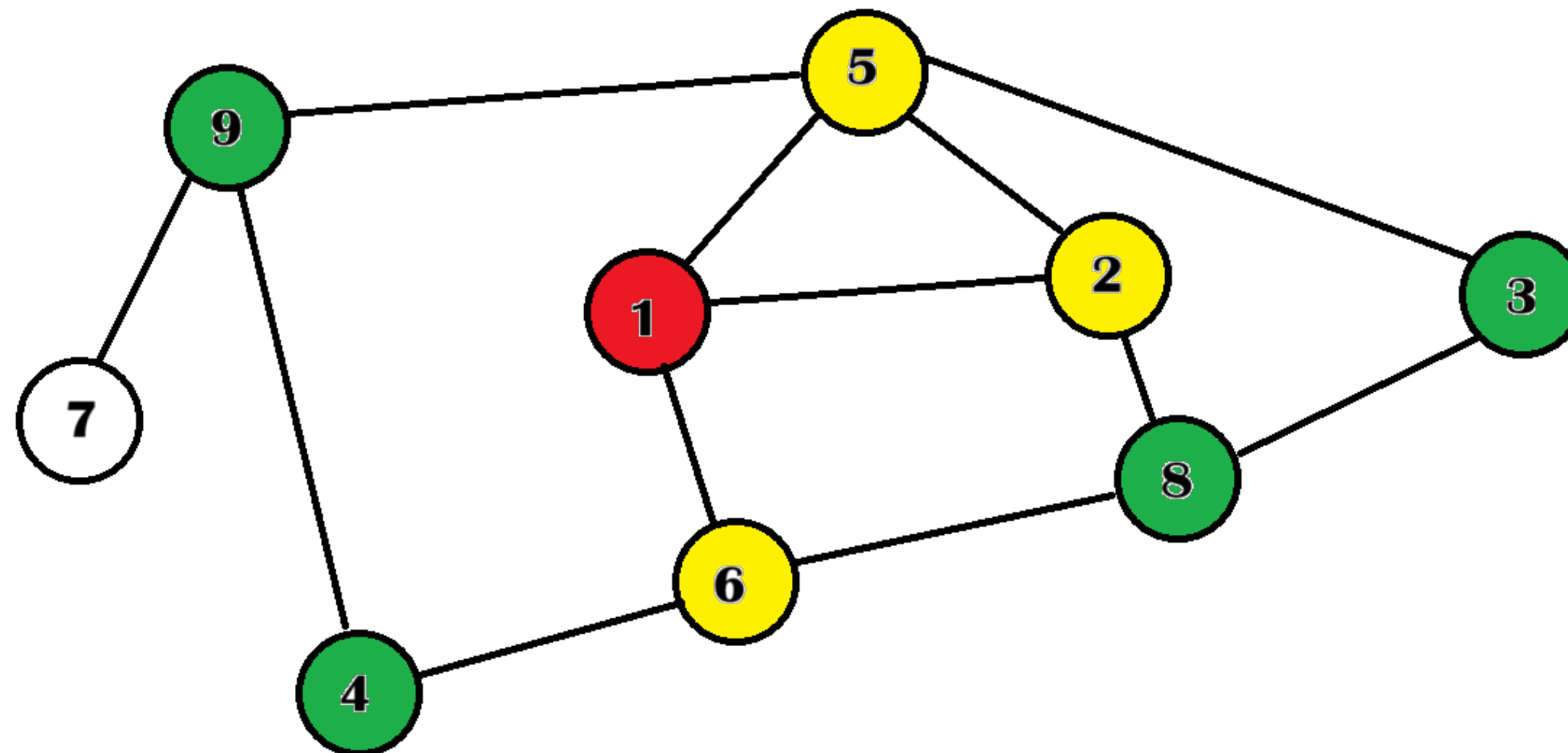
Inicialmente para el único nodo que conozco su distancia es para el inicial, la cual es 0. Digamos que los nodos con distancia 0 son rojos



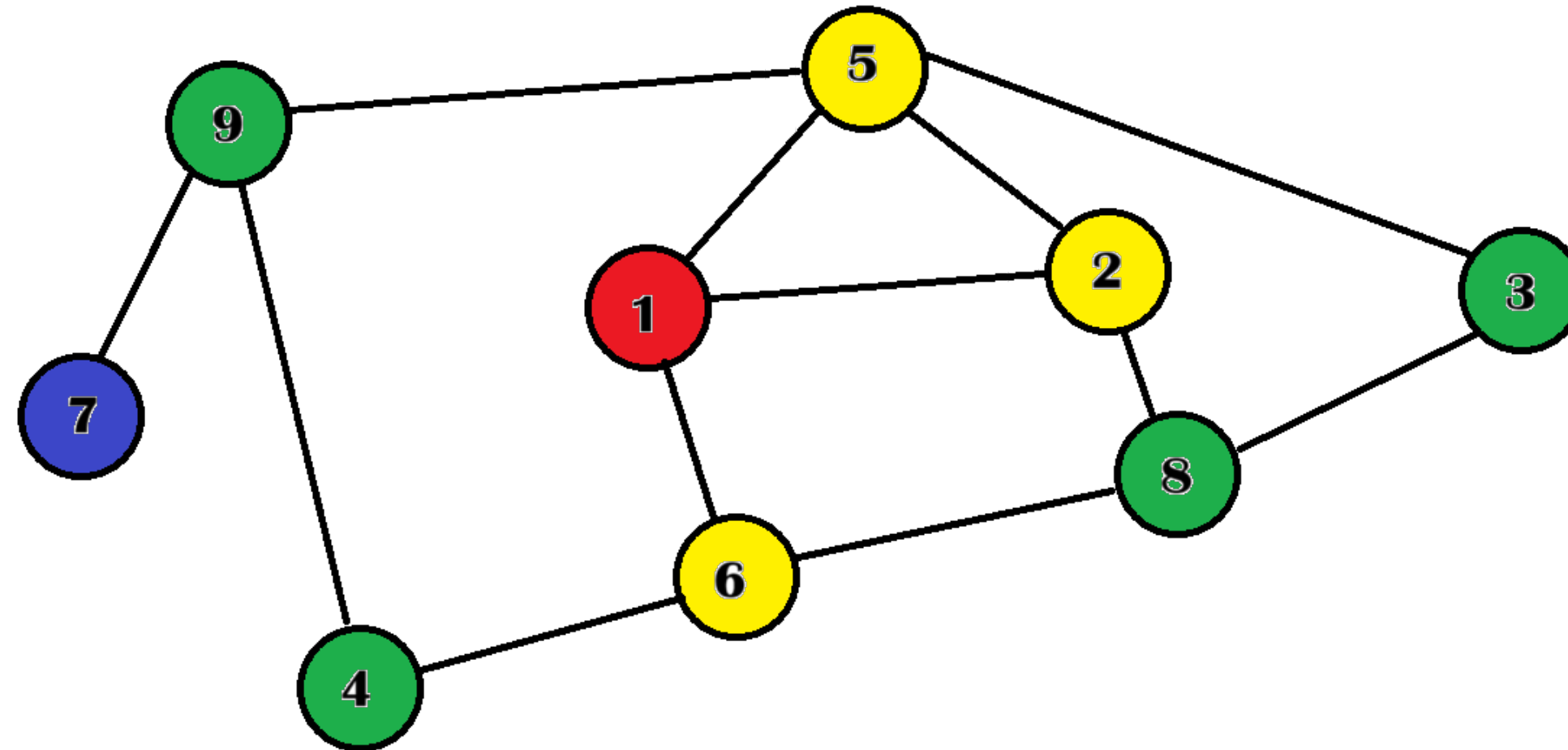
Los nodos con distancia 1 son todos los que comparten una arista con el nodo rojo. Digamos que los nodos con distancia 1 son amarillos



Los nodos con distancia 2 son todos los que comparten una arista con algún nodo amarillo (nodos con distancia 1) y que aún no conseguían una distancia mejor (nodos blancos)



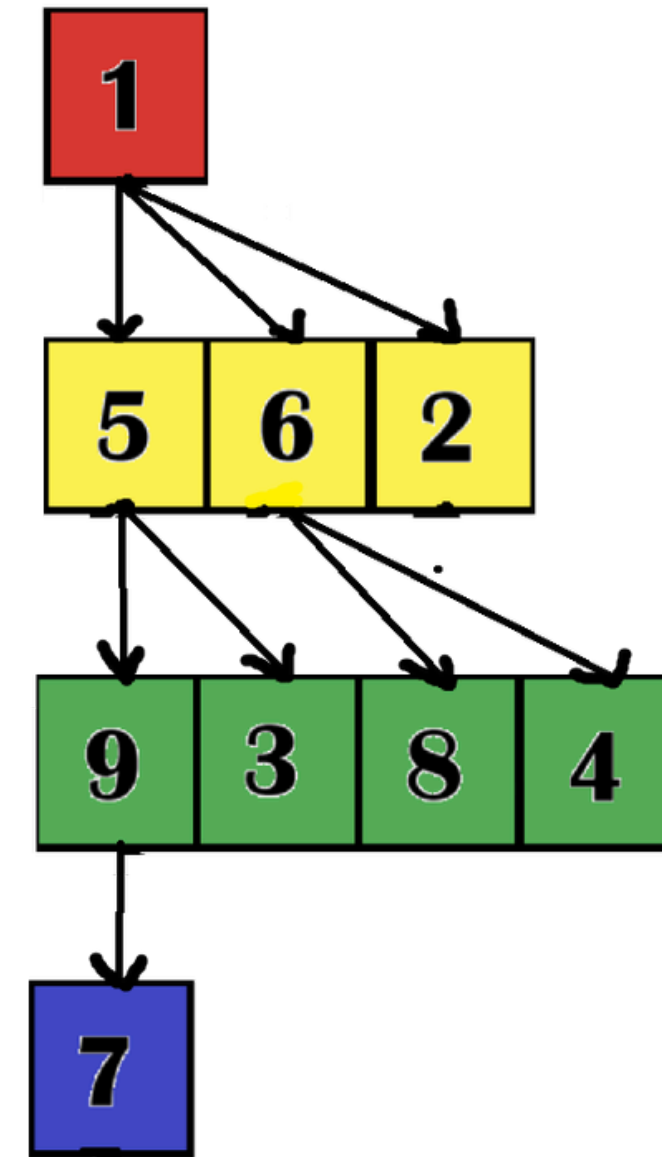
Los nodos con distancia 3 son todos los que comparten una arista con algún nodo verde (nodos con distancia 2) y que aún no conseguían una distancia mejor (nodos blancos)



Y así sucesivamente.....

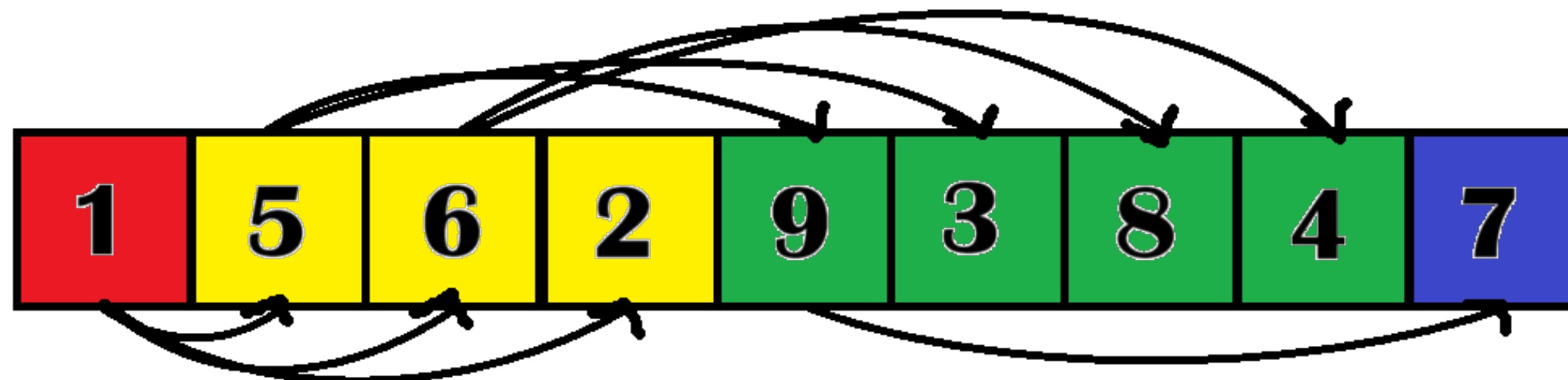
IMPLEMENTACIÓN

Podemos procesar cada color (distancia) uno a uno. Por cada color tenemos un vector con todos los nodos que tienen dicho color. Para encontrar los nodos del siguiente color debemos iterar sobre los nodos del color actual y ver si hay alguna arista hacia un nodo que no había sido visitado antes, entonces dicho nodo pertenece al color siguiente.



MEJORA

De lo anterior podemos notar que los nodos los vamos a ir procesando uno a uno en orden. Además, cada vez que encontramos un nodo nuevo este será procesado justo después de todos los que tenemos pendientes. Así, una forma más sencilla de implementar lo anterior es simplemente usar una cola.



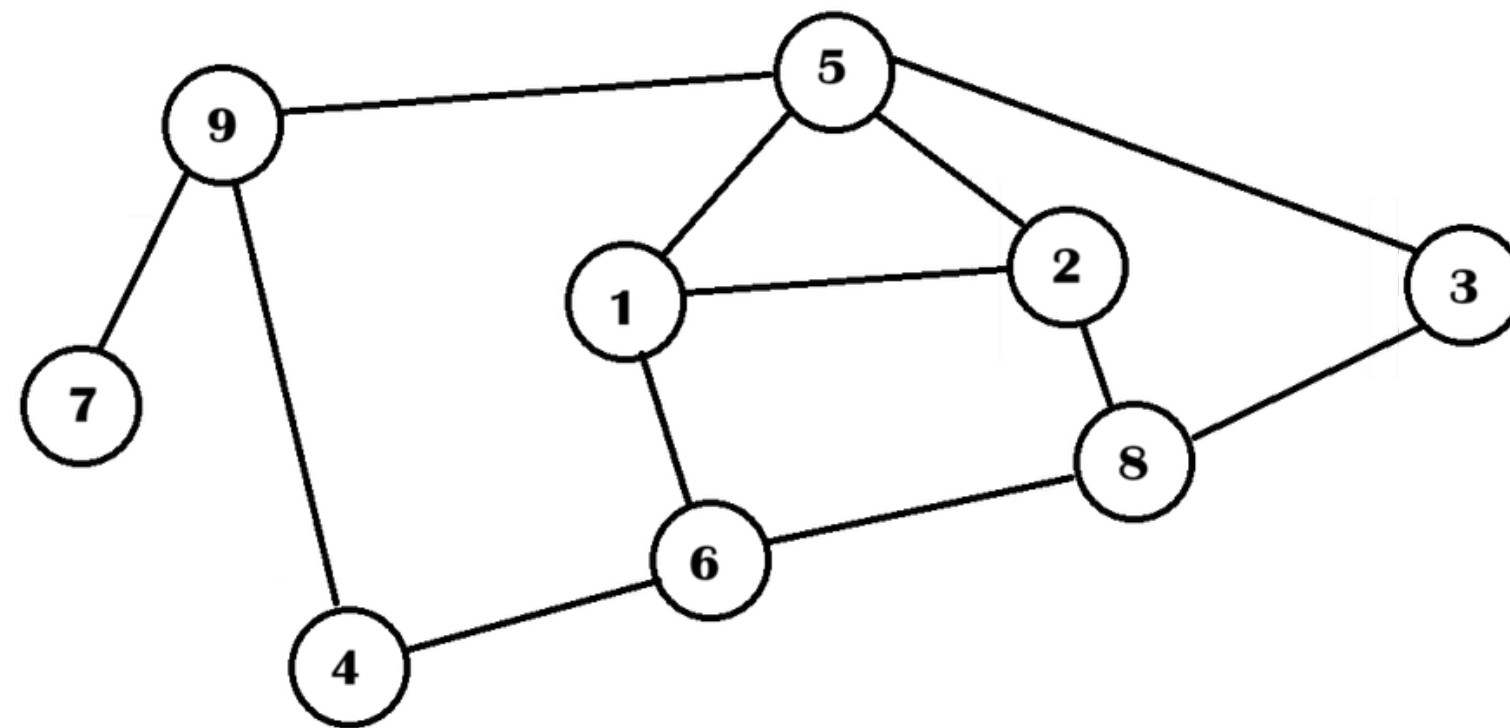
COMPLEJIDAD

Como cada nodo tiene un solo color, solo lo vamos a procesar una vez (cuando toque revisar su correspondiente distancia). Además, cada arista es procesada solo 2 veces, una por cada nodo extremo. Así, la complejidad de este algoritmo es

$$O(n + m)$$

PROBLEMA

Dado un grafo, encuentra un ciclo o indica que no existe ninguno



Por ejemplo, aquí un ciclo sería: 4 - 6 - 8 - 3 - 5 - 9 - 4

SOLUCIÓN

Hagamos BFS desde el nodo 1. Si estoy en un nodo x , el cual es adyacente a un nodo z que ya había sido explorado antes, entonces:

- Existe un camino desde el nodo 1 al nodo x que no usa la arista (x, z)
- Existe un camino desde el nodo 1 al nodo z que no usa la arista (x, z)
- Podemos considerar el camino desde 1 a x , luego usar la arista (x, z) y luego devolvernos por el camino que existe desde z a 1 (que es el mismo que de 1 a z pero invertido)

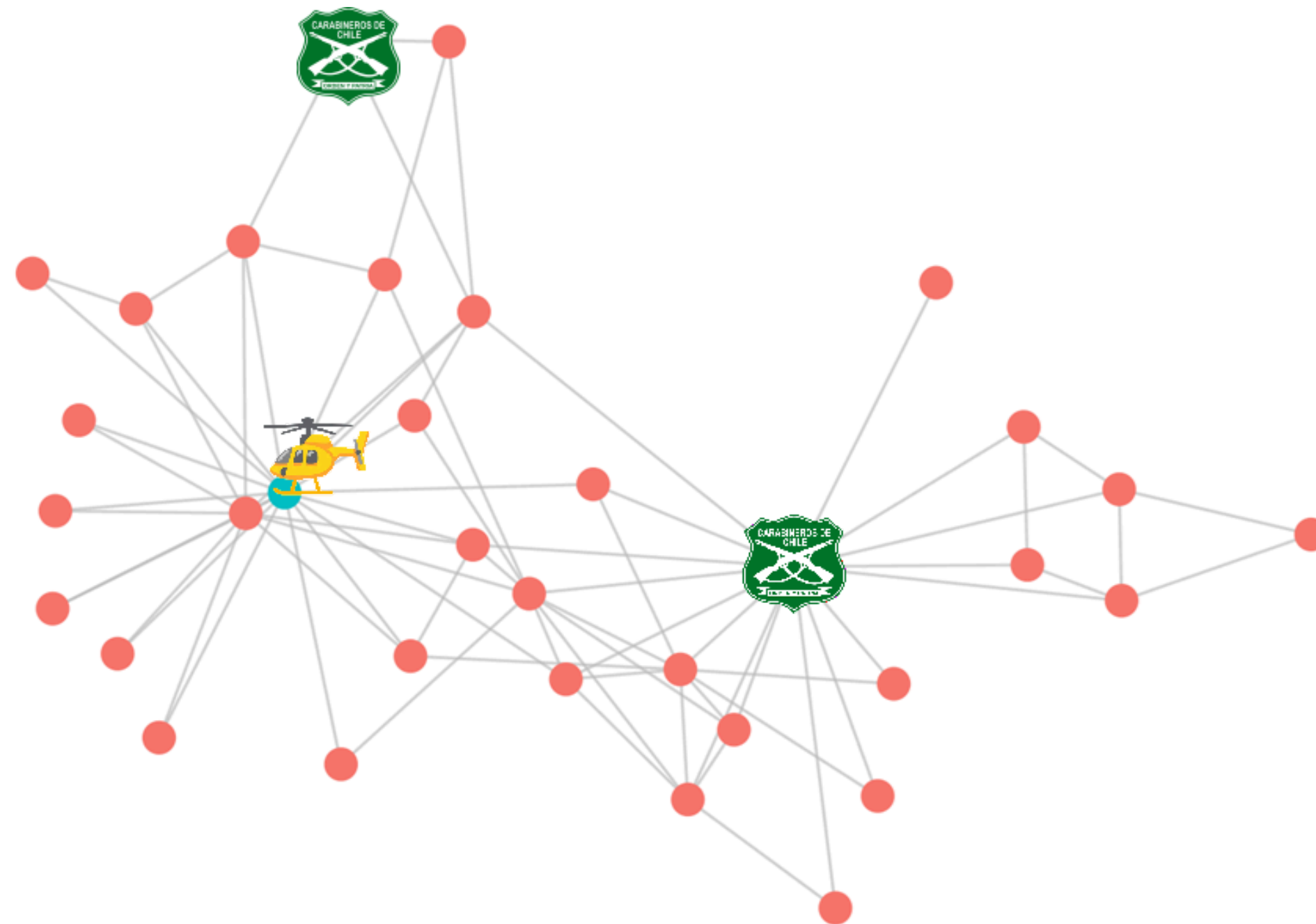
Por lo que para poder implementar lo anterior, además de la distancia debemos guardar el nodo desde el que llegué para poder saber que nodos son partes del ciclo.

PROBLEMA 2

Tenemos una ciudad con n locaciones (nodos) y además sabemos los caminos entre sus locaciones (aristas), recorrer cada camino toma 1 minuto

Un ladrón está planeando un atraco. Sabe en que locaciones están ubicadas las comisarias, además sabe en que locación habrá un helicóptero esperándolo. Quiere saber en que locaciones puede hacer el atraco, esto es, desde que locaciones puede llegar al helicóptero sin que ningún policía lo alcance antes.

PROBLEMA 2



SOLUCIÓN

Podemos encontrar la distancia desde cada nodo a su comisaria más cercana: basta con hacer un BFS, pero en la cola inicialmente ponemos todos los nodos con comisaria a distancia 0.

Podemos encontrar la distancia desde cualquier punto al helicóptero corriendo un simple BFS.

El ladrón puede robar una locación sí y solo sí su distancia al helicóptero es menor que su distancia a la comisaria más cercana.